Universidad Autonoma del Estado de Hidalgo
Licenciatura en Ciencias Computacionales
Automatas y Compiladores
Docente: Dr.Eduardo Cornejo Velazquez

REPORTE DE PRÁCTICA NO.3:

Construcción de un Autómata Finito Determinista

ALUMNO:
Naomi Romero López

1. Introducción

Los autómatas finitos son modelos matemáticos fundamentales en el estudio de la teoría de la computación y los lenguajes formales. Estos modelos permiten representar sistemas computacionales simples que procesan cadenas de símbolos a partir de un conjunto finito de estados y reglas de transición. Su importancia radica en que constituyen la base teórica para diversas aplicaciones en el ámbito computacional, como el diseño de compiladores, analizadores léxicos, reconocimiento de patrones y procesamiento de texto. [1]

Uno de los autómatas más simples de entender es el Autómata Finito Determinista (AFD), el cual se caracteriza porque, para cada estado del sistema y cada símbolo del alfabeto, existe exactamente una transición posible hacia otro estado. Esta propiedad garantiza que el comportamiento del autómata sea completamente predecible para cualquier cadena de entrada. Formalmente, un AFD se define como una quíntupla (, Q, , q0, F), donde representa el alfabeto de entrada, Q es el conjunto finito de estados, es la función de transición, q0 es el estado inicial y F es el conjunto de estados de aceptación [2].

El funcionamiento de un autómata finito determinista consiste en procesar una cadena de símbolos de manera secuencial. El autómata comienza en el estado inicial y, conforme se leen los símbolos de la cadena, realiza transiciones entre estados de acuerdo con la función de transición definida. Al finalizar el procesamiento de la cadena, el autómata determina si esta es aceptada o rechazada, dependiendo de si el estado alcanzado pertenece al conjunto de estados de aceptación. De esta manera, los autómatas finitos permiten reconocer lenguajes regulares, los cuales constituyen la clase más simple dentro de la jerarquía de lenguajes formales[1].

El estudio de los autómatas finitos también es relevante para comprender los fundamentos del comportamiento de sistemas computacionales y la representación formal de problemas relacionados con el reconocimiento de patrones y la validación de cadenas. Según Lewis y Papadimitriou [3], los autómatas finitos proporcionan un marco teórico claro para modelar procesos que dependen de secuencias de entradas y decisiones basadas en estados, lo cual resulta útil en diversas aplicaciones prácticas dentro de la informática y en diferentes áreas de trabajo en general.

En esta práctica se desarrollará un programa utilizando un lenguaje de programación de alto nivel que implemente un Autómata Finito Determinista, con el objetivo de analizar diferentes cadenas de símbolos y determinar si estas pertenecen al lenguaje reconocido por el autómata definido. Para ello, el usuario podrá ingresar en un archivo de texto con un formato especifico los elementos necesarios para poder realizar el proceso. También se utilizarán las estructuras necesarias para representar los estados, el alfabeto, las transiciones y los estados de aceptación, permitiendo evaluar automáticamente si una cadena es aceptada o rechazada por el sistema.

2. Marco teórico

Lenguajes Regulares y Gramáticas Regulares

Los lenguajes regulares son aquellos que pueden ser generados por gramáticas regulares o reconocidos por autómatas finitos. Este tipo de lenguajes resulta especialmente relevante en la construcción de compiladores, ya que los componentes léxicos de un lenguaje de programación pueden expresarse mediante reglas regulares.

Las gramáticas regulares presentan restricciones en la forma de sus producciones, lo que facilita su implementación computacional y permite el uso de modelos equivalentes como los autómatas finitos deterministas y las expresiones regulares.[5]

Autómatas Finitos

Los autómatas finitos son modelos matemáticos utilizados en la teoría de la computación para representar sistemas que procesan secuencias de símbolos de un alfabeto determinado. Estos modelos permiten describir formalmente el comportamiento de sistemas que cambian de estado en respuesta a estímulos externos, lo que los convierte en una herramienta fundamental para el análisis de lenguajes formales y el diseño de sistemas de procesamiento de información. De acuerdo con Michael Sipser, los autómatas finitos constituyen uno de los modelos más simples de computación y son capaces de reconocer una clase específica de lenguajes denominada lenguajes regulares.[1]

Autómata Finito Determinista (AFD)

Un Autómata Finito Determinista (AFD) es un tipo particular de autómata finito en el cual cada estado tiene exactamente una transición definida para cada símbolo del alfabeto de entrada. Esto significa que, para una cadena determinada, el autómata seguirá siempre un único camino de ejecución, lo que hace que su comportamiento sea completamente predecible.

Formalmente, un AFD se define como una quíntupla:

A = (, Q, , q0, F)

donde:

representa el alfabeto de entrada.

Q es el conjunto finito de estados.

es la función de transición que describe el cambio de un estado a otro.

q0 es el estado inicial.

F es el conjunto de estados finales o de aceptación.

Según John E. Hopcroft, Rajeev Motwani y Jeffrey D. Ullman, esta definición formal permite describir de manera precisa el comportamiento de sistemas capaces de reconocer patrones dentro de cadenas de símbolos [2].

Procesamiento de cadenas en un AFD

El funcionamiento de un autómata finito determinista se basa en el procesamiento secuencial de una cadena de entrada. El autómata comienza en el estado inicial y, conforme lee cada símbolo de la cadena, aplica la función de transición correspondiente para determinar el siguiente estado. Este proceso continúa hasta que se han leído todos los símbolos de la cadena.

Al finalizar la lectura de la cadena, el autómata evalúa si el estado alcanzado pertenece al conjunto de estados de aceptación. Si pertenece, se considera que la cadena es aceptada; en caso contrario, la cadena es rechazada. Este mecanismo permite determinar de manera automática si una cadena pertenece o no a un lenguaje formal determinado [3].

3. Herramientas empleadas

Para esta practica fueron utilizados los siguientes recursos de software:

1. Lenguaje de programación Java
2. IDE / Editor: Apache Netbeans

4. Objetivos

Objetivo general

Construir un programa en un lenguaje de programación de alto nivel para implementar un Autómata Finito Determinista (AFD) que permita analizar un conjunto de cadenas de símbolos (W) para determinar si pertenecen o no al lenguaje (L) reconocido por el AFD.

Objetivos específicos

1. Identificar los componentes de un Autómata Finito Determinista (AFD).
2. Implementar un AFD en un lenguaje de programación de alto nivel.
3. Determinar si diferentes conjunto de símbolos (W) pertenece al lenguaje (L) reconocido por el AFD

5. Desarrollo

Para la implementación del Autómata Finito Determinista (AFD) se utilizó el lenguaje de programación Java. El programa fue diseñado para leer la información del autómata desde un archivo de texto y posteriormente evaluar un conjunto de cadenas para determinar si son aceptadas o rechazadas por el autómata definido.

Lectura del archivo de entrada

En primer lugar, se implementó un método para leer el archivo de entrada utilizando la clase BufferedReader junto con FileReader, lo cual permite leer el contenido del archivo línea por línea. Este enfoque facilita el procesamiento de cada sección del archivo de acuerdo con el formato especificado en el enunciado de la práctica.

La primera línea del archivo contiene seis valores enteros que representan los parámetros principales del autómata: el número de estados (N), el tamaño del alfabeto (S), el número de transiciones (D), el estado inicial (q0), el número de estados finales (T) y la cantidad de cadenas a evaluar (C). Estos valores se separan utilizando el método split() y posteriormente se convierten a tipo entero utilizando Integer.parseInt() para poder trabajar con ellos dentro del programa.

Lectura del alfabeto

Posteriormente, se lee la segunda línea del archivo, la cual contiene los símbolos que forman parte del alfabeto del autómata. Estos símbolos también se separan mediante el método split() para almacenarlos en un arreglo de tipo String.

Con el fin de facilitar el acceso a los símbolos del alfabeto durante el procesamiento de las cadenas, se creó una estructura de tipo HashMap que permite asociar cada símbolo con un índice numérico. De esta manera, es posible acceder rápidamente a la columna correspondiente dentro de la tabla de transiciones. Map¡String, Integer¿ indiceSimbolo = new HashMap¡¿(); for (int i = 0; i ¡ S; i++) indiceSimbolo.put(alfabeto[i], i);

Este mapa permite convertir cada símbolo leído de la cadena en una posición dentro de la tabla de transiciones

Estados de aceptación

Después de leer el alfabeto, se procede a leer la línea que contiene los estados de aceptación del autómata. Estos estados se almacenan en un arreglo booleano llamado esFinal, en el cual cada posición indica si un estado determinado es un estado final o no.

Se realiza también una validación para asegurar que el número de estados finales leídos coincida con el valor especificado previamente en la variable T, así como para verificar que los estados se encuentren dentro del rango permitido.

Construcción de la tabla de transiciones

Una vez definidos los estados y el alfabeto, se construye la tabla de transiciones del autómata. Para ello se utiliza una matriz bidimensional trans, donde cada fila representa un estado y cada columna corresponde a un símbolo del alfabeto.

Cada una de las siguientes D líneas del archivo describe una transición del autómata en la forma:

estado$_o rigensímboloestado_d estino$

El programa lee cada transición, valida que el símbolo pertenezca al alfabeto y que los estados se encuentren dentro del rango permitido. Posteriormente, se obtiene el índice del símbolo utilizando el mapa previamente creado y se registra la transición dentro de la matriz.

Procesamiento de las cadenas de entrada

Una vez que el autómata ha sido completamente definido (alfabeto, estados, estados de aceptación y tabla de transiciones), el programa procede a procesar las cadenas de entrada especificadas en el archivo. Estas cadenas representan los posibles elementos del lenguaje que serán evaluados para determinar si pertenecen o no al lenguaje reconocido por el autómata.

Para realizar este proceso, el programa utiliza un ciclo que se ejecuta tantas veces como cadenas hayan sido especificadas en la variable C. En cada iteración se lee una nueva cadena desde el archivo utilizando el objeto BufferedReader. Esta cadena se almacena en una variable de tipo String llamada cadena.

Posteriormente, se inicializa una variable llamada estado, la cual representa el estado actual del autómata durante el procesamiento de la cadena. Esta variable se inicializa con el valor del estado inicial del autómata (q0), ya que todo AFD comienza su ejecución desde dicho estado antes de procesar cualquier símbolo de entrada.

A continuación, el programa recorre la cadena símbolo por símbolo utilizando un ciclo for. En cada iteración se obtiene el carácter correspondiente de la cadena mediante el método charAt() y se convierte en una cadena de tipo String para poder compararlo con los símbolos almacenados en el mapa del alfabeto.

El siguiente paso consiste en verificar si el símbolo leído pertenece al alfabeto definido para el autómata. Esta verificación se realiza utilizando el mapa indiceSimbolo, el cual almacena cada símbolo del alfabeto junto con su índice correspondiente. Si el símbolo no se encuentra dentro de este mapa, significa que no pertenece al alfabeto del autómata, por lo que la cadena se considera inválida. En este caso, el programa asigna el valor -1 al estado actual y detiene el procesamiento de la cadena mediante la instrucción break.

En caso de que el símbolo sí pertenezca al alfabeto, el programa obtiene su índice correspondiente dentro del mapa. Este índice permite identificar la columna correspondiente dentro de la tabla de transiciones del autómata. Utilizando el estado actual y el índice del símbolo, el programa consulta la matriz trans, la cual almacena todas las transiciones del autómata. El valor obtenido representa el nuevo estado al que debe transitar el autómata después de procesar dicho símbolo.

De esta forma, el estado actual se actualiza continuamente conforme se procesan los símbolos de la cadena. Este proceso continúa hasta que todos los símbolos de la cadena han sido leídos o hasta que se detecta un símbolo inválido.

Una vez finalizado el recorrido de la cadena, el programa evalúa si el estado final alcanzado pertenece al conjunto de estados de aceptación del autómata. Esta verificación se realiza consultando el arreglo booleano esFinal, el cual indica qué estados son finales.

Si el estado alcanzado es diferente de -1 y además corresponde a un estado de aceptación, el programa concluye que la cadena pertenece al lenguaje reconocido por el autómata y muestra el mensaje ACEPTADA. En caso contrario, la cadena se considera RECHAZADA.

Este procedimiento se repite para cada una de las cadenas especificadas en el archivo de entrada. Gracias a este proceso, el programa es capaz de simular el comportamiento de un Autómata Finito Determinista,

evaluando cada cadena de entrada y determinando de manera automática si cumple o no con las reglas del lenguaje definido por el autómata.

Algoritmo en pseudocódigo

INICIO

Abrir archivo de entrada

Leer primera línea Separar los valores en: N ← número de estados S ← tamaño del alfabeto D ← número de transiciones q0 ← estado inicial T ← número de estados finales C ← número de cadenas a evaluar

Leer línea del alfabeto Guardar los S símbolos en un arreglo ALFABETO

Crear un mapa para relacionar símbolo → índice

PARA i ← 0 HASTA S-1 HACER indiceSimbolo[ALFABETO[i]] ← i FIN PARA

Leer línea de estados finales Crear arreglo booleano ESFINAL de tamaño N+1

PARA cada estado leído HACER ESFINAL[estado] ← VERDADERO FIN PARA

Crear matriz TRANSICION de tamaño (N+1) × S

PARA i ← 1 HASTA D HACER Leer transición: origen, símbolo, destino

col ← indiceSimbolo[símbolo]

TRANSICION[origen][col] ← destino FIN PARA

// Procesar cadenas

PARA i ← 1 HASTA C HACER

Leer cadena estado ← q0

PARA cada símbolo de la cadena HACER

SI símbolo NO pertenece al alfabeto ENTONCES estado ← -1 SALIR del ciclo FIN SI

col ← indiceSimbolo[símbolo]

estado ← TRANSICION[estado][col]

FIN PARA

SI estado -1 Y ESFINAL[estado] = VERDADERO ENTONCES Imprimir ”ACEPTADA” SINO Imprimir ”RECHAZADA” FIN SI

FIN PARA

Cerrar archivo

FIN

6. Resultados

En esta sección se presentan los resultados obtenidos al ejecutar el programa que implementa el Autómata Finito Determinista (AFD). Para realizar las pruebas, se utilizó un archivo de entrada que contiene la definición del autómata, incluyendo el número de estados, el alfabeto, las transiciones, los estados de aceptación y las cadenas que deben evaluarse.

El programa lee la información del archivo, construye internamente la estructura del autómata y posteriormente analiza cada una de las cadenas de entrada para determinar si pertenecen o no al lenguaje reconocido por el autómata.

Durante la ejecución del programa se observó que el sistema procesa correctamente cada cadena, realizando las transiciones entre estados de acuerdo con la tabla definida. Al finalizar el procesamiento de cada cadena, el programa muestra un mensaje indicando si la cadena es ACEPTADA o RECHAZADA, dependiendo de si el estado final alcanzado pertenece al conjunto de estados de aceptación.

En la Figura 1 se observa el archivo ingresado para las pruebas

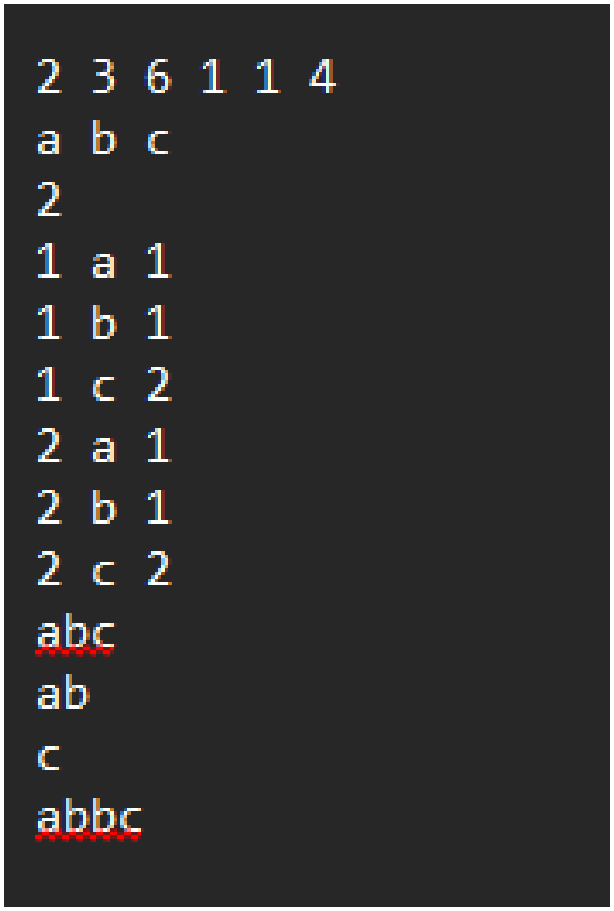

Figure 1: Archvio para pruebas ingresado

Siguiendo la logica que las cadenas deben de terminar en "bc" estos fueron los resultados presentados, en la Figura 2 se observa el comportamiento del programa ante los ejemplos ingresados para la confirmación de las reglas. En la Figura 3 se observa el codigo utilizado para comprobar si los ejemplos son aceptados o

```
tput - Automata_Finito_Determinista (run)
run:
abc ACEPTADA
ab RECHAZADA
c ACEPTADA
abbc ACEPTADA
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 2: Reconocimiento de los ejemplos ingresados

rechazados.

```
for (int i = 0; i < C; i++) {

    String cadena = br.readLine();
    int estado = q0;

    for (int j = 0; j < cadena.length(); j++) {

        String simbolo = String.valueOf(c: cadena.charAt(index: j));

        if (!indiceSimbolo.containsKey(key: simbolo)) {
            estado = -1;
            break;
        }

        int col = indiceSimbolo.get(key: simbolo);
        estado = trans[estado][col];
    }

    if (estado != -1 && esFinal[estado]) {
        System.out.println(cadena + " ACEPTADA");
    } else {
        System.out.println(cadena + " RECHAZADA");
    }
}
```

Figure 3: Reconocimiento palabras en minúsculas

Conclusiones

Durante el desarrollo de esta práctica se logró implementar un Autómata Finito Determinista (AFD) utilizando el lenguaje de programación Java, lo que permitió comprender de manera práctica cómo funcionan estos modelos teóricos dentro del área de la teoría de la computación. A través de la lectura de un archivo de entrada, el programa fue capaz de interpretar los componentes fundamentales del autómata, como el conjunto de estados, el alfabeto, las transiciones, el estado inicial y los estados de aceptación.

El proceso de desarrollo permitió analizar cómo un autómata procesa una cadena de símbolos de manera secuencial, realizando transiciones entre estados de acuerdo con las reglas definidas en la función de transición. De esta forma, fue posible observar cómo el autómata determina si una cadena pertenece o no al lenguaje que reconoce, dependiendo del estado final alcanzado después de procesar todos los símbolos de la cadena.

Asimismo, la implementación del programa permitió reforzar conceptos importantes relacionados con el manejo de estructuras de datos, como arreglos, matrices y mapas, los cuales fueron utilizados para representar el alfabeto, los estados de aceptación y la tabla de transiciones del autómata. Esto facilitó la simulación del comportamiento del AFD dentro del programa.

Finalmente, esta práctica permitió comprender la importancia de los autómatas finitos dentro del campo de la informática, ya que estos modelos constituyen la base teórica para diversas aplicaciones, como el diseño de compiladores, el reconocimiento de patrones y el procesamiento de lenguajes formales. La implementación del autómata en un lenguaje de programación permitió conectar los conceptos teóricos con su aplicación práctica dentro del desarrollo de software.

Referencias Bibliográficas

References

[1] Sipser, M. (**2013**). *Introduction to the Theory of Computation* (3.rd ed.). Cengage Learning. ISBN: 978-1133187790.

[2] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (**2006**). *Introduction to Automata Theory, Languages, and Computation* (3.rd ed.). Pearson. ISBN: 978-0321455369.

[3] Lewis, H. R., & Papadimitriou, C. H. (**1997**). *Elements of the Theory of Computation* (2.nd ed.). Prentice Hall. ISBN: 978-0132624781.

[4] Mishra, K. L. P. (**2006**). *Theory of Computer Science: Automata, Languages and Computation*. PHI Learning. ISBN: 978-8120329687.

[5] Ruiz Catalán, J. (**2010**). *Compiladores: Teoría e implementación*. Alfaomega. ISBN: 978-607-7854-68-5.